

# Chapter 25: ViewChild

## 1. Overview

A component's template is not just markup — it is a graph of child components, directives, and DOM elements that the parent frequently needs a *handle* on: to call an imperative method on a child ( `chart.redraw()` ), to read a native element's dimensions, to focus an input after a view update, or to react whenever the set of matched nodes changes (e.g., items rendered by `*ngFor` ).

`@ViewChild` / `@ViewChildren` (and their modern replacements `ViewChild()` / `ViewChildren()` ) are Angular's mechanism for **querying the view DOM** — the elements and components a component's *own template* renders — as opposed to **content queries** ( `@ContentChild` / `ContentChild()` ), which look at what was **projected in** via `<ng-content>` from the parent's usage site. This chapter focuses on view queries; content projection queries are covered in depth in the Content Projection chapter, but the API surface is nearly identical, so drawing the parallel here matters for interviews.

The central interview tension in this topic is **timing**: queries only resolve after Angular has run change detection over the relevant part of the view, and static vs. dynamic queries resolve at different lifecycle points. Getting this wrong is one of the most common real-world Angular bugs ( `ViewChild is undefined in ngOnInit` ), and it is a near-universal interview question.

With Angular 17+, the signal-based `ViewChild()` / `ViewChildren()` functions supersede the decorator API for new code: queries become signals, integrate into the reactive graph, support `computed()` derivations, and eliminate the entire "did `ngAfterViewInit` fire yet" class of bugs by exposing `undefined` type-safely instead of silently failing.

## 2. Core Concepts

### 2.1 What a view query can target

A `@ViewChild` / `ViewChild()` selector can be:

- **A component or directive type** — `@ViewChild(ChildComponent)` — matches the first instance of that component/directive in the view.
- **A template reference variable** — `@ViewChild('myRef')` — matches the element or directive/component tagged `#myRef` in the template.
- **A string token / `InjectionToken`** — matches a provider registered on that node.
- `ElementRef` as the `read` option — to force resolution to the native DOM element instead of the component instance, even when the selector is a component type.

```
<child-comp #childRef></child-comp>
<div #plainDiv>Some markup</div>
```

```
@ViewChild('childRef') childCompInstance!: ChildComponent;
@ViewChild('childRef', { read: ElementRef }) childElement!: ElementRef;
@ViewChild('plainDiv') plainDivRef!: ElementRef<HTMLDivElement>;
```

## 2.2 Single vs. multiple: ViewChild vs. ViewChildren

- `@ViewChild` / `ViewChild()` returns the **first** match (or `undefined` /throws if none).
- `@ViewChildren` / `ViewChildren()` returns **all** matches as a `QueryList<T>` (decorator API) or a readonly `Signal<ReadonlyArray<T>>` (signal API).

## 2.3 `static: true` vs. `static: false` — the timing contract

This is the single most tested nuance of the decorator API.

```
@ViewChild('ref', { static: true }) staticRef!: ElementRef;  
@ViewChild('ref', { static: false }) dynamicRef!: ElementRef; // default
```

- `static: true` tells Angular: "this node is *not* inside a structural directive (`*ngIf`, `*ngFor`, etc.) — it is guaranteed to exist as soon as the component's own view is created." Angular resolves it **before** `ngOnInit` runs, so it is safe to read inside `ngOnInit` (and even the constructor-adjacent lifecycle, though `ngOnInit` is the documented safe point).
- `static: false` (the default since Angular 9) means "this node's existence may depend on bindings/structural directives, so resolve it only after the full view (including child components' own change detection) has been checked." It is resolved **right before** `ngAfterViewInit` and re-resolved on every subsequent change detection pass where the view changes, with `ngAfterViewChecked` firing after each re-check.
- If the queried element sits inside `*ngIf` / `*ngFor`, `static: true` is **invalid** — Angular cannot guarantee it exists before the first CD pass, and setting it anyway can yield stale or incorrect references (in older Angular this was actually enforced at compile time in some scenarios; today it's a semantic contract you must uphold).
- Signal-based `ViewChild()` **has no static option at all** — it always behaves like dynamic resolution but exposes the value as a signal that starts `undefined` and becomes populated once Angular sets it, sidestepping the static/dynamic split entirely (see §4.3).

## 2.4 QueryList — the decorator API's collection type

`@ViewChildren` returns a `QueryList<T>`, not a plain array. Key facts:

- It implements `Iterable<T>`, so `for...of`, spreading, `.toArray()`, `.map()`, `.filter()`, `.forEach()`, `.first`, `.last`, `.length` all work.
- It is **immutable from the outside** — you cannot push/splice it; Angular replaces its internal contents when the query re-resolves.
- It exposes a `.changes: Observable<QueryList<T>>` that emits **every time the matched set changes** (items added/removed/reordered by `*ngFor`, `*ngIf` toggling elements in/out, etc.). It does **not** emit when an existing item's own internal state changes — only when the *set of matched nodes* changes.
- `QueryList` itself is only fully populated **after** `ngAfterViewInit`, same dynamic-resolution timing as any `static:false` query, because Angular can't know the final count until change detection has run over `*ngFor` / `*ngIf`.

## 2.5 Signal-based queries — `viewChild()` / `viewChildren()`

Introduced as stable in Angular 17.3/18, these are functions called in field initializers of a component class:

```
childRef = viewChild<ElementRef<HTMLInputElement>>('inputRef');    //
Signal<ElementRef|undefined>
childRequired = viewChild.required<ChildComponent>(ChildComponent); //
Signal<ChildComponent> (throws if absent)
allItems = viewChildren(ItemComponent);                             //
Signal<ReadonlyArray<ItemComponent>>
```

Differences from decorators:

- They are **signals**, so they can feed directly into `computed()`, `effect()`, and templates without manual subscription/unsubscription plumbing.
- No `static` option — resolution timing is handled uniformly and safely; reading the signal before the view is ready simply yields `undefined` (or throws for `.required()`), which TypeScript forces you to handle.
- `viewChildren()` returns a plain readonly array signal, not a `QueryList` — no `.changes` Observable; instead, derive with `computed()` or use `effect()` to react to changes, which is simpler and more idiomatic in a signals-first codebase.
- Because they're plain class fields (not decorated properties needing metadata), they compose better with standalone components and are tree-shake/AOT friendlier.

## 2.6 Brief contrast: `ContentChild/ContentChildren` vs. `ViewChild/ViewChildren`

- **ViewChild** queries nodes defined in **this component's own template** (the view DOM Angular generates for the component).
- **ContentChild** queries nodes **projected into** this component from its *usage site* via `<ng-content>` (the transcluded/content DOM).
- Timing differs too: content queries resolve in `ngAfterContentInit` / `ngAfterContentChecked`, one lifecycle phase *earlier* than view queries (`ngAfterViewInit` / `ngAfterViewChecked`), because Angular processes content before it processes the component's own view. (Full treatment — including `descendants` option — lives in the Content Projection chapter.)

## 2.7 Accessing child component API surface

Once resolved, a `@ViewChild(ChildComponent)` reference is simply the child's **class instance** — you can call any public method or read/write any public property on it directly, imperatively:

```
this.childCompInstance.refreshData();
this.childCompInstance.title = 'Updated';
```

This is a deliberate escape hatch from the normal unidirectional `@Input` / `@Output` data flow — useful for imperative APIs (opening a modal, resetting a form, focusing an element) but easy to overuse into tightly-coupled parent/child spaghetti; prefer `@Input` / `@Output` / services for anything that should stay declarative.

## 2.8 ElementRef and direct DOM access

`ElementRef.nativeElement` gives raw access to the underlying DOM node. Angular deliberately keeps this API thin and discourages direct manipulation:

- Direct DOM writes (`nativeElement.style.color = 'red'`, `nativeElement.innerHTML = ...`) bypass Angular's change detection and sanitization, and **will throw or silently no-op in non-browser rendering contexts** (Server-Side Rendering, Web Workers) because there is no real DOM there.
- The Angular team's official guidance is to use `Renderer2` (or the newer `afterRenderEffect` /host bindings) for any DOM manipulation, since `Renderer2` is platform-abstracted and safe under SSR/Universal, and centralizes sanitization.

## 3. Code Examples

### 3.1 Decorator-based: static vs. dynamic, component and template-ref queries

```
import {
  Component, ViewChild, ViewChildren, ElementRef, QueryList,
  AfterViewInit, OnInit, AfterViewChecked
} from '@angular/core';

@Component({
  selector: 'app-child',
  template: `<p>{{ label }}</p>`,
})
export class ChildComponent {
  label = 'child';
  greet(): string {
    return `Hello from ${this.label}`;
  }
}

@Component({
  selector: 'app-parent',
  template: `
    <div #staticDiv>Always present</div>

    <div *ngIf="showConditional" #conditionalDiv>Conditionally present</div>

    <app-child #childRef></app-child>

    <ul>
      <li *ngFor="let item of items" #itemRef>{{ item }}</li>
    </ul>
  `,
})
export class ParentComponent implements OnInit, AfterViewInit, AfterViewChecked {
  showConditional = false;
  items = ['a', 'b', 'c'];
```

```

// Not inside *ngIf/*ngFor -> guaranteed to exist at view-creation time.
@ViewChild('staticDiv', { static: true }) staticDiv!: ElementRef<HTMLDivElement>;

// Inside *ngIf -> existence depends on a binding; must be dynamic (the default).
@ViewChild('conditionalDiv') conditionalDiv?: ElementRef<HTMLDivElement>;

@ViewChild('childRef') childComp!: ChildComponent;

// Multiple matches from *ngFor -> QueryList.
@ViewChildren('itemRef') itemRefs!: QueryList<ElementRef<HTMLLIElement>>;

ngOnInit(): void {
  console.log(this.staticDiv.nativeElement.textContent); // OK: resolved already
  console.log(this.conditionalDiv);                      // undefined: showConditional is
false
  console.log(this.childComp);                          // undefined: view not yet
checked
}

ngAfterViewInit(): void {
  console.log(this.childComp.greet());                   // OK now
  console.log(this.itemRefs.length);                     // 3, resolved after view init

  // Mutating a query-bound value inside ngAfterViewInit throws
  // ExpressionChangedAfterItHasBeenCheckedError in dev mode unless deferred:
  this.showConditional = true;
  Promise.resolve().then(() => {
    // re-run CD in a microtask so the dynamic query settles cleanly
    console.log(this.conditionalDiv); // will populate on next check
  });
}

ngAfterViewChecked(): void {
  // Runs after every CD pass over the view – including query re-resolution.
}
}

```

## 3.2 QueryList.changes subscription

```

import { Component, AfterViewInit, ViewChildren, QueryList, OnDestroy } from
'@angular/core';
import { Subscription } from 'rxjs';

@Component({
  selector: 'app-list',
  template: `
    <button (click)="add()">Add</button>
    <div *ngFor="let n of numbers" #rowRef>{{ n }}</div>
  `,
})
export class ListComponent implements AfterViewInit, OnDestroy {
  numbers = [1, 2, 3];
}

```

```

@viewChildren('rowRef') rows!: QueryList<any>;
private sub!: Subscription;

add(): void {
  this.numbers = [...this.numbers, this.numbers.length + 1];
}

ngAfterViewInit(): void {
  // Fires the first time synchronously-available data is read; then again
  // every time *ngFor adds/removes a matched row.
  this.sub = this.rows.changes.subscribe((list: QueryList<any>) => {
    console.log('Row count changed:', list.length);
  });
}

ngOnDestroy(): void {
  this.sub.unsubscribe(); // QueryList.changes never auto-completes on destroy
}
}

```

### 3.3 Signal-based queries

```

import { Component, viewChild, viewChildren, effect, computed, ElementRef } from
'@angular/core';

@Component({
  selector: 'app-signal-parent',
  standalone: true,
  template: `
    <input #searchBox />
    <app-child #childRef></app-child>
    <li *ngFor="let x of items" #rowRef>{{ x }}</li>
  `,
})
export class SignalParentComponent {
  items = ['x', 'y', 'z'];

  // Signal<ElementRef | undefined> – safe, no static/dynamic distinction to worry about.
  searchBox = viewChild<ElementRef<HTMLInputElement>>('searchBox');

  // Throws synchronously if never resolved – use only when existence is guaranteed.
  childRef = viewChild.required<ChildComponent>();

  rows = viewChildren<ElementRef>('rowRef');
  rowCount = computed(() => this.rows().length);

  constructor() {
    effect(() => {
      // Re-runs automatically whenever the underlying query result changes –
      // no manual subscribe/unsubscribe, no QueryList needed.
      console.log('Row count is now', this.rowCount());
    });
  }
}

```

```

});

effect(() => {
  const input = this.searchBox();
  if (input) {
    input.nativeElement.focus();
  }
});
}
}

```

## 3.4 Calling a child method only after the view is initialized

```

@Component({ /* ... */ })
export class HostComponent implements AfterViewInit {
  @ViewChild(ChatComponent) chat!: ChatComponent;

  ngAfterViewInit(): void {
    // Correct place: the child's own view has been created and checked.
    this.chat.redraw();
  }

  onWindowResize(): void {
    // Also safe here, any time after ngAfterViewInit has fired once.
    this.chat.redraw();
  }
}

```

## 4. Internal Working

### 4.1 Query metadata compiled into the component definition

Ivy compiles `@ViewChild/@viewChildren` (and `ViewChild()/viewChildren()`) into **view query instructions** embedded in the component's generated definition (`@cmp`), not into runtime reflection metadata as View Engine once did. Concretely, the compiler emits calls like `ɵɵviewQuery(selector, flags, read)` inside a special `viewQuery` function attached to the component definition, plus `ɵɵqueryRefresh()` and `ɵɵloadQuery()` calls that run during the update pass.

### 4.2 Two-pass resolution: creation vs. update

Every component's `viewQuery` function runs twice per change-detection cycle, mirroring the general create/update pass structure of Ivy's rendering model:

1. **Creation pass** (`RenderFlags.Create`): Angular registers the query — i.e., declares "look for nodes matching this selector within this view" — walking the `LView` data structure for the component.
2. **Update pass** (`RenderFlags.Update`): Angular calls `ɵɵqueryRefresh()`, which checks whether the query's underlying results (tracked in an internal `LQueries/TQueries` structure attached to the view's `TView`) have changed since last check. If dirty, it calls `ɵɵloadQuery()` and assigns the new value(s) to the component instance's field — this is the moment your `@ViewChild` property or `QueryList` actually gets (re-)populated.

Static queries are special-cased: because the compiler can statically prove the node isn't behind a structural directive, Angular resolves them during the **first creation pass itself**, before `ngOnInit` fires. Dynamic queries can only be resolved during an **update pass**, and specifically only after Angular has walked into and checked child views/embedded views (i.e., after `*ngIf`/`*ngFor` have materialized or removed their content) — which is why the earliest safe read point is `ngAfterViewInit`, the lifecycle hook Angular invokes immediately after the first full update pass over the view completes.

### 4.3 Why `ngAfterViewChecked` fires repeatedly and `QueryList` re-emits

Because dynamic/`QueryList`-based queries are re-evaluated on **every** change detection pass (not just the first), Angular calls `ngAfterViewChecked` after each check, and — if the *matched set itself* changed (not merely a property on an existing match) — the query's dirty flag is set, `QueryList` is refreshed with a new backing array, and its `.changes` `EventEmitter` (a thin wrapper Angular fires manually, it is not a plain RxJS `Subject` under the hood but behaves like one) emits synchronously as part of that same CD cycle, right after `ɵɵqueryRefresh` completes for that query.

### 4.4 Signal-based queries and the reactive graph

`ViewChild()` / `ViewChildren()` are implemented on the same underlying Ivy query instructions (`ɵɵviewQuery`) as the decorators — the compiler still emits the same `TQueries` bookkeeping. The difference is purely in how the *result* is exposed to user code: instead of eagerly writing to a plain class property, Ivy writes the resolved value into a `WritableSignal` backing the query function. Reading `mySignal()` inside a `computed()` or `effect()` registers a dependency the same way any other signal read does, so:

- Consumers don't need lifecycle hooks at all — an `effect()` registered in the constructor will simply not "fire meaningfully" (or read `undefined`) until Ivy pushes the first resolved value into the signal, then re-fires automatically once the value changes — no `ngAfterViewInit` boilerplate needed.
- This also means query resolution timing is *observably* the same as dynamic decorator queries (still gated on view creation/update passes) — signals don't make the DOM appear earlier, they just remove the ceremony of subscribing/hook-timing around reading it.

## 5. Edge Cases & Gotchas

- **undefined in `ngOnInit` for dynamic queries.** The single most common bug report: a `@ViewChild` (default, `static: false`) read inside `ngOnInit` or the constructor is always `undefined`, because the view hasn't been created/checked yet. Fix: move the read to `ngAfterViewInit`, or mark it `static: true` *only if* the node is unconditionally present in the template.
- **`*ngIf` hiding the queried element.** If the target element is behind `*ngIf="false"` at init time, the query resolves to `undefined` even in `ngAfterViewInit` — and it will only populate once the condition flips true *and* a subsequent CD pass runs. With the decorator API this requires either re-checking in `ngAfterViewChecked` (careful — it fires very often) or subscribing to `QueryList.changes` for multi-element cases. The signal API handles this more gracefully: the signal is just `undefined` until it isn't, and an `effect()` will pick up the change automatically without extra plumbing.
- **Mutating component state that affects a query, inside `ngAfterViewInit`, causes `ExpressionChangedAfterItHasBeenCheckedError` in dev mode.** Because `ngAfterViewInit` runs *during* the first CD pass, changing a bound value there (which then changes what a query matches)



conflicts with Angular's single-pass-per-check invariant. Defer with `Promise.resolve().then(...)`, `setTimeout`, or restructure to avoid mutating template-affecting state in this hook.

- **QueryList going "stale" without subscribing to `.changes`**. If code captures `this.itemRefs.toArray()` once (e.g., in `ngAfterViewInit`) and the underlying `*ngFor` list later adds/removes elements, the captured array is a **snapshot** — it does not stay in sync. You must either re-read `this.itemRefs` fresh each time, or subscribe to `.changes` and only trust the array delivered in the callback.
- **Forgetting to unsubscribe from `QueryList.changes`**. Unlike some other Angular observables, `.changes` does not automatically complete on component destroy — a subscription left open across component instances (e.g., inside a service, or captured in a closure that outlives the component) is a genuine memory leak. Always unsubscribe in `ngOnDestroy`, or better, use `takeUntilDestroyed()` from `@angular/core/rxjs-interop`.
- **@viewChildren used where @ViewChild was intended (or vice versa)**. Querying a component type present multiple times in the template with `@viewChild` silently returns only the first match — a frequent source of "why is my second instance not updating" bugs.
- **read option confusion**. `@ViewChild(ChildComponent)` gives you the component instance; `@ViewChild(ChildComponent, { read: ElementRef })` gives you its host DOM element instead. Mixing these up produces confusing runtime errors like "property does not exist on ElementRef."
- **Direct DOM manipulation via `ElementRef.nativeElement` breaks SSR**. Code like `this.el.nativeElement.focus()` or reading `.offsetWidth` executes fine in the browser but throws or is a no-op under Angular Universal/SSR, because there is no real layout engine on the server. Guard such calls with `isPlatformBrowser(this.platformId)`, or better, replace them with `Renderer2` calls / the `afterRenderEffect` / `afterNextRender` APIs which are SSR-aware by design.
- **Direct DOM writes bypass sanitization and change detection**. Setting `nativeElement.innerHTML` directly skips Angular's built-in XSS sanitization (unlike `[innerHTML]` binding) and can leave the view out of sync with component state, causing subtle rendering bugs on the next CD cycle.
- **static: true on a node that's actually conditional**. If a developer marks a query `static: true` for a node that is (or later becomes, after a refactor) wrapped in `*ngIf`, the query can resolve to a stale/incorrect reference or `undefined` at the wrong time — this contract is not always caught by the compiler, so it's a classic code-review-miss / interview trap.
- **Signal query `.required()` throwing during SSR or early construction**. `viewChild.required(...)` throws synchronously if read before Ivy has resolved it — same "too early" trap as `ngOnInit` decorator reads, just surfaced as a thrown error instead of silent `undefined`. Only use `.required()` when you are certain of the read timing (e.g., inside `ngAfterViewInit`-equivalent code or a later `effect()`).

## 6. Interview Questions & Answers

---

### Q1. What is the difference between `@ViewChild` and `@viewChildren`?

`@ViewChild` returns the first element/component/directive matching the selector (or `undefined`), while `@viewChildren` returns a `QueryList` containing *all* matches. Use `@ViewChild` when you expect exactly one instance (a single child component, a single template ref), and `@viewChildren` when the template can render a variable number of matches, typically via `*ngFor`.

## Q2. Why is a `@ViewChild` property undefined when read inside `ngOnInit`?

**Interviewer intent:** checks whether the candidate actually understands Angular's lifecycle/query resolution timing rather than having memorized "use `static: true`" as a magic incantation.

By default (`static: false`), Angular can only resolve a view query after it has fully created and checked the component's view — which includes evaluating structural directives like `*ngIf`/`*ngFor` that might affect whether the target node even exists. `ngOnInit` runs *before* the view has been created, so the query hasn't run yet and the property is `undefined`. The property becomes populated only right before `ngAfterViewInit` fires (and is kept in sync on each subsequent view-check pass thereafter). The fix is either to read it in `ngAfterViewInit`, or, if the node is guaranteed to be unconditionally present in the template, mark the query `static: true` so Angular resolves it during view creation, before `ngOnInit`.

## Q3. When is it valid to use `static: true`, and when is it dangerous?

It's valid only when the queried node is **guaranteed to exist regardless of any binding or structural directive** — e.g., a plain `<div #ref>` with no `*ngIf`/`*ngFor` wrapping it. It's dangerous on any node inside `*ngIf`, `*ngFor`, `*ngSwitch`, or any conditionally-rendered child component, because Angular can't guarantee the node exists at view-creation time; using `static: true` there can yield `undefined` or a stale reference with no clear error signal, since the compiler doesn't always catch every such case reliably across all template shapes.

## Q4. What is a `QueryList`, and how is it different from a plain array?

`QueryList<T>` is Angular's live collection type for multi-match queries (`@ViewChildren`/`@ContentChildren`). It's iterable and supports array-like helpers (`map`, `filter`, `forEach`, `toArray`, `first`, `last`), but it is immutable from outside code — you cannot push/splice into it directly; Angular replaces its contents internally whenever the query re-resolves. Crucially, it exposes a `.changes` Observable that emits every time the set of matched elements changes (items added/removed/reordered), which a plain array has no equivalent for.

## Q5. How do you react to `*ngFor`-driven items being added or removed, if you need the updated child references?

Subscribe to `QueryList.changes` in `ngAfterViewInit` (the query list itself is only populated by then): `this.items.changes.subscribe(list => ...)`. Remember to store the `Subscription` and unsubscribe in `ngOnDestroy` (or use `takeUntilDestroyed()`), since `.changes` doesn't auto-complete on component destroy. With the signal-based `viewChildren()` API, this is unnecessary — just derive a `computed()` from the signal or read it inside an `effect()`, and it updates automatically.

## Q6. How would you get a reference to the native DOM element of a component queried via `@ViewChild(ChildComponent)`?

`@ViewChild(ChildComponent)`?

Pass the `read` option: `@ViewChild(ChildComponent, { read: ElementRef }) el!: ElementRef;`. Without `read`, querying by component type returns the **component instance**, not its element; `read: ElementRef` forces Angular to resolve to the host DOM node instead.

## Q7. Why does Angular discourage direct DOM manipulation via `ElementRef.nativeElement`, and what should you use instead?

**Interviewer intent:** probes whether the candidate has hit real production issues around SSR/Universal or security, not just theoretical awareness.

Two concrete reasons: (1) SSR/Universal — there's no real DOM on the server, so code that calls `nativeElement.focus()`, reads `offsetWidth`, or mutates `style` directly will throw or no-op when rendered server-side, unless explicitly guarded with `isPlatformBrowser()`. (2) Bypassing Angular's abstractions — direct writes like `innerHTML = ...` skip Angular's built-in sanitization (a genuine XSS risk compared to `[innerHTML]` bindings) and leave the rendered DOM out of sync with the component's own change-detection

cycle, causing state drift on the next CD pass. Angular's official recommendation is `Renderer2` for imperative DOM work (it's platform-abstracted, so it degrades safely under SSR) or, for post-render DOM reads/writes tied to change detection, the newer `afterNextRender` / `afterRenderEffect` APIs.

**Q8. Explain the difference between `ngAfterViewInit` and `ngAfterViewChecked`, in relation to view queries.**

`ngAfterViewInit` fires exactly once, immediately after Angular has created and performed its *first* full check of the component's view (including child components) — this is the earliest safe point to read a dynamic (`static: false`) `@ViewChild`. `ngAfterViewChecked` fires after **every** subsequent change-detection pass over the view, whether or not anything actually changed in the query results — so it's the hook where you'd notice a query has been *re-resolved* to a new value, but it's also expensive to put heavy logic in, since it runs on every CD cycle.

**Q9. How does `ViewChild()` (signal-based) differ from `@ViewChild` in terms of the `static` option?**

There is no `static` option in the signal API at all. Signal queries always resolve using the same underlying dynamic timing as `static: false` decorator queries — Ivy just pushes the resolved value into a `writableSignal` once the corresponding view-check pass completes, rather than eagerly assigning a class property that a lifecycle hook then reads. The upside is type safety: the signal's type is `Signal<T | undefined>` (or `Signal<T>` with `.required()`, which throws if unresolved), so the compiler forces you to handle the "not yet resolved" case instead of silently allowing an uninitialized non-null-asserted property (!) to slip through.

**Q10. Can you call a public method on a child component obtained via `@ViewChild`? What's the tradeoff versus `@Input` / `@Output`?**

Yes — once resolved, the queried reference is the actual child instance, so `this.child.someMethod()` or `this.child.someProperty = x` work directly, imperatively. The tradeoff is coupling: `@Input` / `@Output` keep data flow declarative and the parent/child contract explicit and testable in isolation; reaching into a child via `@ViewChild` to call arbitrary methods creates a tighter, more implicit coupling that's harder to mock/test and can violate the component's encapsulation if overused. It's appropriate for genuinely imperative needs (triggering an animation, opening a modal, resetting a form, focusing an element) but shouldn't replace normal reactive data binding.

**Q11. What happens if you query for a component type that appears zero times, or more than once, in the template using `@ViewChild`?**

Zero matches: the property is `undefined` (decorator API) or the signal returns `undefined` (`ViewChild()`) / throws (`ViewChild.required()`). More than one match: `@ViewChild` silently returns only the **first** match in document order — there is no warning that additional matches exist. This is a common source of bugs when a template is refactored to render a component multiple times (e.g., wrapped in `*ngFor`) but the query wasn't updated from `@ViewChild` to `@viewChildren`.

**Q12. Contrast `@ViewChild` and `@ContentChild` — what's the practical difference and why does it affect lifecycle timing?**

**Interviewer intent:** verifies the candidate understands Angular's content-vs-view separation, a concept many mid-level developers gloss over because both APIs "look the same."

`@ViewChild` queries nodes rendered by the component's **own template** — content the component itself owns and fully controls. `@ContentChild` queries nodes **projected in** from the parent via `<ng-content>` — content the component doesn't own, only hosts. Because Angular processes projected content *before* it processes the host component's own view (content has to be evaluated in the context of the parent first, since `<ng-content>` merely relocates already-created nodes), content queries resolve one phase earlier:

`ngAfterContentInit` / `ngAfterContentChecked`, versus `ngAfterViewInit` / `ngAfterViewChecked` for view queries. Reading a `@ContentChild` too early (e.g. expecting view-query timing) is a subtler variant of the same "resolved too early" bug.

### Q13. Why would reading a query result trigger `ExpressionChangedAfterItHasBeenCheckedError`, and how do you avoid it?

If you mutate a property inside `ngAfterViewInit` (or `ngAfterContentInit`) that affects what a *later-checked* binding renders — including indirectly changing what a query would match — Angular's dev-mode double-check catches the mismatch between what was rendered in this pass and what would render if checked again, because you mutated state mid-pass. The practical fix is to defer the mutation to a microtask/macrotask (`Promise.resolve().then(...)`, `setTimeout`), move the logic earlier (before the view is checked, if the timing allows), or use `ChangeDetectorRef.detectChanges()` / `markForCheck()` explicitly to force a controlled second pass instead of relying on Angular's implicit one.

### Q14. How would you focus an input element as soon as it becomes visible after an `*ngIf` toggles to true, using both APIs?

With decorators: query it as `dynamic (@ViewChild('inp') inp?: ElementRef)`, and since it only exists after the `*ngIf` flips, you can't rely on `ngAfterViewInit` alone — you'd typically trigger the focus from the same event handler that flips the boolean, after awaiting a microtask/ `setTimeout(0)` so Angular has re-run CD and the element exists, or use `ngAfterViewChecked` guarded by a "already focused" flag to avoid infinite/duplicate calls. With signals: declare `inp = viewChild<ElementRef>('inp')` and register `effect(() => { const e1 = this.inp(); if (e1) e1.nativeElement.focus(); })` — the effect automatically re-runs the moment the signal transitions from `undefined` to a real value, with no manual timing/flag bookkeeping required. This is a good example of how the signal API structurally eliminates a whole category of manual-timing bugs.

## 7. Quick Revision Cheat Sheet

- `@ViewChild(sel)` → first match; `@ViewChildren(sel)` → `QueryList<T>` of all matches. View queries look at the component's **own template**; content queries (`@ContentChild` / `@ContentChildren`) look at **projected** `<ng-content>` nodes.
- Selector can be: component/directive type, template ref string (`'#ref'`), `InjectionToken` / string DI token. Use `{ read: x }` to force resolution to a different token (e.g., `ElementRef` instead of the component instance).
- `static: true` → resolved before `ngOnInit`; valid **only** if the node is unconditionally present (not inside `*ngIf` / `*ngFor`).
- `static: false` (default) → resolved right before `ngAfterViewInit`, re-resolved on every CD pass thereafter; safe read point is `ngAfterViewInit` / later, never `ngOnInit` / constructor.
- `QueryList`: iterable, has `.toArray()`, `.first`, `.last`, `.length`; immutable from outside; `.changes` Observable emits when the *matched set* changes — remember to unsubscribe (`ngOnDestroy` / `takeUntilDestroyed()`).
- Signal API: `viewChild()` / `viewChildren()` — no `static` option, returns `Signal<T|undefined>` / `Signal<ReadOnlyArray<T>>`; `.required()` variant throws if read unresolved; integrates with `computed()` / `effect()`, no manual subscription needed.
- Internally: Ivy compiles queries into `ɵviewQuery` instructions on the component def; resolved via a creation pass (static) or update pass (`ɵqueryRefresh` / `ɵloadQuery`, dynamic) walking

`TQueries / LQueries`. Signal queries use the same instructions, writing into a `writableSignal` instead of a plain field.

- Content queries resolve one phase earlier (`ngAfterContentInit`) than view queries (`ngAfterViewInit`), because projected content is processed before the host's own view.
- Common gotchas: `undefined` in `ngOnInit`; `*ngIf`-hidden targets resolving late; stale captured `QueryList.toArray()` snapshots; leaked `.changes` subscriptions; `ExpressionChangedAfterItHasBeenCheckedError` from mutating query-affecting state in `ngAfterViewInit`; direct `nativeElement` DOM writes breaking SSR and bypassing sanitization — prefer `Renderer2 / afterNextRender`.

**Created By - Durgesh Singh**